

**AI Chatbot using
Transformers
A PROJECT REPORT
FOR
Introduction to AI (AI101B)
Session (2024-25)**

Submitted By

Shashank Singh (202410116100194)

Tarun Kumar (202410116100223)

Ved Teotia (202410116100238)

Vishal Dadar (202410116100249)

**Submitted in the partial fulfilment
of the Requirements of the Degree of**

MASTER OF COMPUTER APPLICATION

Under the Supervision of

Mr. Apoorv Jain

Assistant Professor



**Submitted to
DEPARTMENT OF COMPUTER APPLICATIONS
KIET Group of Institutions, Ghaziabad
Uttar Pradesh-201206
(MAY - 2025)**

DECLARATION

We hereby declare that the work presented in this project entitled “**AI Chatbot Using Transformers**” was carried out by us. We have not submitted the matter embodied in this report for the award of any other degree or diploma of any other University or Institute.

We have given due credit to the original authors/sources for all the words, ideas, diagrams, graphics, computer programs, experiments, results, that are not my original contribution. We have used quotation marks to identify verbatim sentences and given credit to the original authors/sources.

We affirm that no portion of my work is plagiarized, and the experiments and results reported in the report are not manipulated. In the event of a complaint of plagiarism and the manipulation of the experiments and results, we shall be fully responsible and answerable.

Shashank Singh (202410116100194)
Tarun Kumar (202410116100223)
Ved Teotia (202410116100238)
Vishal Dagar (202410116100249)

CERTIFICATE

Certified that Shashank Singh, Ved Teotia, Tarun Kumar, Vishal Dagar has carried out the project work having “**AI Chatbot Using Transformers**” (INTRODUCTION TO AI) (AI101B) for Master of Computer Application from Dr. A.P.J. Abdul Kalam Technical University (AKTU) (formerly UPTU), Lucknow under my supervision.

The project report embodies original work, and studies are carried out by the student themselves and the contents of the project report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution. This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Mr. Apoorv Jain

Assistant Professor

Department of Computer Applications

KIET Group of Institutions, Ghaziabad

An Autonomous Institutions

Dr. Akash Rajak

Dean

Department of Computer Applications

KIET Group of Institutions, Ghaziabad

An Autonomous Institutions

ABSTRACT

Artificial Intelligence (AI) has transformed the way humans interact with machines, with chatbots emerging as a key application in automating conversations and improving user engagement. This project focuses on developing an intelligent chatbot using Transformer-based architectures to deliver more human-like and context-aware responses. Transformers, especially models like BERT and GPT, have revolutionized natural language processing by enabling deep contextual understanding of text.

The chatbot is trained and fine-tuned on relevant conversational datasets to enhance its language understanding and generation capabilities. Key preprocessing steps such as tokenization, lowercasing, and handling special characters are implemented to improve model input quality. The Transformer model leverages multi-head self-attention mechanisms and positional encoding to capture context over long conversations, ensuring coherent and relevant replies.

To evaluate the chatbot's effectiveness, metrics such as perplexity, BLEU score, response relevance, and user satisfaction are considered. The results indicate that the Transformer-based chatbot significantly outperforms traditional rule-based or retrieval-based models in producing natural and meaningful dialogue. Furthermore, the model is deployed using Streamlit, offering an interactive web interface where users can engage with the chatbot in real time.

This study highlights the potential of Transformer-based AI systems in enhancing conversational AI applications, paving the way for more intuitive and efficient human-computer interactions.

ACKNOWLEDGEMENT

First and foremost, I would like to express my sincere gratitude to my project supervisor, **Mr. Apoorv Jain**, for their continuous support, expert guidance, and encouragement throughout this project. Their insight and feedback were invaluable at every stage.

Words are not enough to express my gratitude to Dr. Akash Rajak, Professor and Dean, Department of Computer Applications, for his insightful comments and administrative help on various occasions.

Fortunately, I have many understanding friends, who have helped me a lot on many critical conditions.

Finally, my sincere thanks go to my family members and all those who have directly and indirectly provided me with moral support and other kind of help. Without their support, completion of this work would not have been possible in time. They keep my life filled with enjoyment and happiness.

Shashank Singh (202410116100194)

Tarun Kumar (202410116100223)

Ved Teotia (202410116100238)

Vishal Dagar (202410116100249)

TABLE OF CONTENTS

1. Declaration	2
2. Certificate.....	3
3. Abstract	4
4. Acknowledgement.....	5
5. Table of contents	6
6. Introduction... ..	7-9
7. Literature review	10-11
8. Project description... ..	12-15
9. Methodology	16-18
a. Data Collection	16
b. Feature Extraction	16
c. Visualization and Tools.....	18
10. Code implementation	19-27
a. Code structure and usage.....	19
b. Core functionalities implemented	20
c. Code.....	22-27
11. Code explanation.....	28-31
12. Data analysis... ..	32-33
13. Output explanation	34-39
14. Future enhancements	40-41
15. Conclusion.....	42
16. References	43

1 Introduction

The landscape of human-computer interaction has been irrevocably transformed by the emergence of chatbots. These software entities, often manifestations of artificial intelligence, are designed to engage in conversations with humans across various messaging platforms. From simplifying customer service operations to aiding in information retrieval, chatbots have become indispensable tools in the modern digital ecosystem. Moreover, the ubiquity of virtual assistant gadgets like Alexa and Google Home underscores the growing demand for AI-driven conversational agents. These devices, equipped with advanced natural language processing capabilities, exemplify the potential of chatbots to seamlessly integrate into the daily lives. Yet, beneath the surface of these virtual assistants lies a profound question: How do these machines comprehend and generate human-like responses? This question serves as the foundation of constructing a chatbot entirely from scratch. Unlike many existing solutions that rely on predefined APIs, the approach is characterized by its bespoke nature, requiring us to build each component from the ground up. One critical architectural choice that defines the approach is the adoption of the Transformer architecture. This architectural paradigm, introduced in the seminal paper "Attention is All You Need" by Vaswani et al. (2017), has revolutionized the field of natural language processing. The Transformer's capacity to model complex, long-range dependencies in text through self-attention mechanisms presents an ideal framework for the chatbot's development. While alternative models like BERT and GPT, which also leverage the Transformer architecture, have garnered significant attention in NLP tasks, the focus remains on building a robust, versatile chatbot.

The impetus for this research was a challenge posed on Kaggle, a platform renowned for hosting data science competitions. Kaggle provided access to a dataset that served as the foundation for the chatbot's training and evaluation. As I shall elucidate in subsequent sections, this dataset encompasses a diverse array of conversational exchanges, making it an ideal crucible for the development of a chatbot capable of engaging in human-like dialogues. It involves navigating the intricacies of

Seq2Seq (sequence- to-sequence) modeling, a paradigm that aligns seamlessly with the nature of chatbot interactions. I recognize that the crux of a successful chatbot lies in its ability to handle sequences of input and generate coherent sequences of output, a fundamental characteristic of conversation. Among the available architectural choices, I contend that the Transformer-based approach, with its inherent self-attention mechanisms, best equips us to tackle this challenge. In the subsequent sections of this paper, I will embark on a comprehensive exploration of the chatbot's development. I will delve into the theoretical foundations of the Transformer architecture, elucidate the practical aspects of data preprocessing and model training, and ultimately present the findings and results. The endeavor underscores the potential of combining diverse AI techniques, including Logistic Regression and Deep Learning, to create a chatbot that not only comprehends user intent but also generates contextually relevant and coherent responses.

2 Approaches for Training

In my pursuit of creating an effective chatbot, I explored several training approaches, each with its unique architecture and capabilities. These approaches are pivotal in understanding the evolution of my chatbot and its enhancing conversational capabilities. Below, I provide an overview of the three primary training approaches employed in my research.

Vanilla Encoder-Decoder model based on the principles of sequence-to-sequence (Seq2Seq) modeling, served as a starting point for my chatbot's evolution. In this architecture, I designed three distinct classes, each extending the TensorFlow `tf.keras.Model` class, a technique known as Model subclassing in TensorFlow. Despite its simplicity, the Vanilla Encoder-Decoder model boasted a substantial number of trainable parameters, totaling 23 million. During training, I executed 20 epochs, although learning ceased to improve after just seven epochs. This limitation was a notable shortcoming of this initial model iteration. The learning curves generated by this model exhibited a depressing trend, underscoring the inherent challenges in training a chatbot. Perhaps the most critical concern with the Vanilla Encoder-Decoder

model was the quality of responses generated by the chatbot. Regrettably, the chatbot's replies were marred by incoherence and gibberish, highlighting the considerable room for improvement in capturing context and producing meaningful dialogues. Recognizing the limitations of the Vanilla Encoder-Decoder model, my research ventured further by incorporating an LSTM-based Encoder-Decoder architecture with a Bahdanau attention mechanism. This enhancement aimed to imbue the chatbot with improved context understanding and response generation capabilities. In this architecture, the fundamental structure remained consistent with the Vanilla model, encompassing three classes that extended the `tf.keras.Model` class. However, the addition of the Bahdanau attention mechanism introduced a level of complexity and sophistication that was absent in the initial model. This iteration resulted in a model with 27.5 million trainable parameters. The training process consisted of 20 epochs, similar to the Vanilla model. Although learning ceased after seven epochs, the LSTM-based Encoder-Decoder model demonstrated superior performance compared to its predecessor. It exhibited an enhanced ability to generate responses that, while not perfect, exhibited greater coherence and relevance. However, there remained a perplexing issue during training, with the model persistently generating responses such as "I am not sure," which fell short of expectations and left room for further improvement.

In response to the persistent challenges faced in earlier model iterations, I made a transformative leap by adopting the Transformer architecture. The Transformer model, first introduced in 2017, marked a revolutionary shift in natural language processing. At its core, the Transformer leveraged self-attention mechanisms to effectively capture long-range dependencies in text, making it an ideal candidate for my chatbot's development. The decision to embrace the Transformer architecture represented a pivotal moment in my research. I recognized that the self-attention capabilities of the Transformer would enable my chatbot to model complex linguistic relationships and generate more contextually relevant responses. The details of the Transformer implementation are extensive and form the crux of my research, underscoring its significance in the

development of my chatbot. With 24 million trainable parameters and a substantially longer training process of 350 epochs, the Transformer model exhibited remarkable learning capabilities, even if somewhat slow. The results were highly promising, with the chatbot generating responses that showed traces of intelligence and coherence. This marked a significant milestone in my research, demonstrating the power of the Transformer architecture and its potential to elevate chatbot capabilities beyond the limitations of previous approaches. Creating a sophisticated and context-aware chatbot had taken a transformative turn, setting the stage for future enhancements and applications. In the subsequent sections, I delve into the intricacies of the Transformer architecture, detailing its components, training strategies, and the results achieved. My exploration not only showcases the effectiveness of the Transformer but also opens doors to exciting possibilities for the future of conversational AI.

3 Literature Review

Within this section, an encompassing exploration of contemporary contributions is presented, categorized into four distinct paradigms: Rule-based, Retrieval-based, Learning-based, and Generative-based.

3.1 Rule-Based Models

The realm of rule-based models initiates with the definition of characteristic variables extracted from input expressions, culminating in the provision of predefined responses based on these variables and established rules. Rule-based approaches bifurcate into two principal categories: pattern matching methods and standard task-oriented systems. In pattern matching, Chatbots scrutinize user inputs against rule patterns, electing predetermined responses through pattern matching algorithms. Task-oriented systems, conversely, steer users through predefined actions, exemplified by Chatbots designed for specific domains such as restaurant and film reservations, a concept dating back to the inception of the Eliza Chatbot. Eliza utilized pattern-based analysis reliant on parsing rules embedded within sentence keywords. Subsequent advancements, as seen in "Pari," have incorporated influential variables like "fear,"

”anger,” and ”distrust,” ushering in more naturalistic conversations. The deployment of the Artificial Intelligence Markup Language (AIML) in ALICE further exemplifies rule-based methodologies. Additionally, platforms like Microsoft LUIS, IBM Watson Assistant, and Dialogflow have been developed to facilitate Chatbot creation. Despite their simplicity and cost-effectiveness, rule-based Chatbots confront issues of inflexibility, limited learning capabilities, and confinement to predefined.

3.2 Retrieval-Based

Retrieval-based Chatbots differentiate themselves by selecting the most fitting response to a user's query from a pre-constructed conversational repository. Initial approaches, such as Lowe et al.'s TF-IDF-based model, computed TF-IDF vectors for question-answer pairs and identified the response with the highest cosine similarity. Subsequent advancements leveraged neural network architectures like Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU). For example, Zhou et al. introduced an attention mechanism within deep networks to enhance question-answer matching, while Shu et al. proposed a two-stage transformer-based retrieval model with keyword extraction modules. Retrieval-based methods, widely adopted by researchers, excel in selecting the most probable answer from a database but face limitations in generating entirely novel responses and rely on the availability of a comprehensive database during inference.

3.3 Learning-Based

The learning-based approach predominantly centers on the Seq2Seq learning model, a formidable solution for handling lengthy sentences and conversations exceeding 20 words. Researchers have devised various strategies to address the challenge of compressing extensive source sentence information into a fixed-length vector. These strategies encompass structural modifications like incorporating word embedding matrices, encoder or decoder adjustments, and the integration of attention mechanisms. Some researchers have embraced transformer-based models as alternatives, supplanting traditional recurrent layers in encoder-decoder architectures with multi-headed self-attention. Rao et al. pioneered a hybrid model employing the Elmo language model for embedding, Variational Autoencoder (VAE), and Bi-directional Long Short-Term Memory (BiLSTM). Additionally, models like DIALOGPT, a transformer-based answer generation model by Zhang et al., and BERT-based Chatbot models have demonstrated promising outcomes. While learning-based models exhibit

significant potential, they grapple with limitations such as their inability to generate diverse responses due to inherent constraints in learning the answer distribution.

3.4 Generative-Based

Generative-based methods revolutionize response generation by learning answer distributions through generative models, including SeqGAN and StepGAN, often employing Reinforcement Learning (RL) techniques. SeqGAN employs Monte Carlo Tree Search (MCTS) to optimize discriminator feedback, while StepGAN evaluates the Generative Adversarial Network (GAN) in a step-wise manner, enabling the discriminator to assign scores to generated subsequences. Innovations like the addition of the Truth guidance loss function and discriminator networks based on self-attention mechanisms have further enriched generative models. Notably, a Transformer-based Implicit Latent GAN (TILGAN) model combines transformer autoencoders with GANs, introducing novel designs and distribution matching using the Kullback-Leibler (KL) divergence. In the current landscape of chatbot development, the success of ChatGPT has spurred further exploration of possibilities, though several challenges persist in accurately emulating human conversation, especially within specific domains. To address these challenges, this paper proposes a novel architecture founded on cWGAN and the transformer model, offering parallel processing capabilities during training to overcome sequential constraints and enhance response accuracy.

4 Data

The inception of the transformer model in 2017 marked a pivotal moment in the field of natural language processing, ushering in a revolutionary paradigm shift. Unlike its predecessors, the transformer model dispensed with the chronological dependencies necessitated by recurrent neural networks, opting instead for a straightforward feed-forward neural network. At first glance, the concept of 'self-attention' may appear formidable, but it simplifies into a series of matrix multiplications, making it accessible even to those new to the technology. The fundamental building blocks of the transformer model are grounded in simplicity: feed-forward neural networks, residual connections, layer normalization, positional encodings, and masking. In this section, I delve into the data preparation process, a crucial initial step in harnessing the power of the transformer.

4.1 Data Preparation

With pandas data frame, I selectively extract two pivotal columns: 'question' and 'answer.' Subsequently, a meticulous cleansing process ensues, involving the removal of extraneous special characters, de-contraction of contractions, and other necessary data pre-processing steps. This meticulous data refinement paves the way for the partitioning of the data set into distinct train and validation data frames. Understanding that neural networks possess an affinity for numerical representations, I embark on the process of tokenization. Subword tokenization, despite its somewhat esoteric nomenclature, serves as the mechanism through which each word metamorphoses into a numerical entity, a transformation that aligns the data with the neural network's preferences. In summary, this section provides a detailed account of the foundational data preparation steps undertaken to ensure compatibility between the data set and the transformer model, thereby setting the stage for subsequent model development and training.

5 Model

The core architecture of the proposed model draws inspiration from the foundational concepts of Generative Adversarial Networks, comprising two

fundamental components: the generator and the discriminator. In the context of this paper, the GAN is uniquely designed to process questions in the form of sequences, denoted as x . The discriminator is trained to maximize the score $D(x, y^*)$ while minimizing the score $D(x, y)$, where y^* represents the ground truth answer. This optimization process compels the generator to produce an answer y that maximizes $D(x, y)$ as articulated in the below equation:

$$\min_{\theta} \max_{\phi} E[\log D(x, y^*)] + E[\log(1 - D(x, y))]$$

Here, (x, y^*) represents the joint probability distribution of (x, y) , with $P(x)$ denoting the probability distribution of x derived from the training data. While the GAN framework offers substantial capabilities, it can sometimes encounter issues such as non-convergence and mode collapse. Researchers have proposed various GAN variants to address these challenges. One notable variant is the Wasserstein GAN, an extension that introduces an alternative approach to training the generator. Unlike the traditional GAN, which employs a discriminator to classify data as real or fake, WGAN replaces the discriminator with a critic. This critic assigns scores to the authenticity of the data, aiming to minimize the distribution gap between the data observed in the training data set and the generated examples. This approach enhances training stability, particularly when working with gradients. In light of the demonstrated advantages of WGAN over traditional GANs, I present a novel architecture that combines the conditional WGAN with a transformer model for answer generation in Chatbots. The rationale behind incorporating WGAN into the proposed method lies in its ability to operate without the constraints imposed by the sigmoid activation function, which typically limits output values to 0 or 1.

1. Moreover, the training process exhibits greater stability when I fuse WGAN with the transformer. proposed architecture, illustrated in Figure 2, consists of two primary modules: the generator and the discriminator, intricately interconnected within a single network. The adoption of WGAN within the framework serves to bolster training stability and enhance the overall performance of the model.

5.1 Generator Modules

The generator is implemented as a complete transformer model responsible for generating synthetic answers during the testing phase. Figure 3 provides an overview of the generator architecture during the training phase, drawing inspiration from previous work. In preparation for training, the encoder and decoder input embeddings are constructed. To process the encoder input, I tokenize and embed real questions using a pre-trained BERT model featuring 12 layers, 768 features, and 12 heads. To expedite training, I employ a linear model to reduce feature dimensionality, defined by the below equation.

$$y = f_{hspace0.1cm}(\sum_i w_i x_i + \theta)$$

Following this dimensionality reduction, the output of the linear model is concatenated with positional encoding to retain word positions within the sentence. Positional encoding, a crucial matrix, provides context based on word position, as outlined in by the below equations.

$$PE(pos, 2i) = \sin(pos/10000^{(2i/d_{model})})$$

$$PE(pos, 2i + 1) = \cos(pos/10000^{(2i/d_{model})})$$

Here, "pos" signifies the position of a word within the sequence, "d" represents the embedding size, and "i" spans the dimensions of the embedding. Similar preprocessing steps are undertaken for the decoder; however, instead of a question, the decoder takes a real answer as input. The transformer operates differently during training and inference phases. During inference, only a question is provided as an input sequence, with no actual answer as a target sequence for the decoder. Given that the decoder aims to generate an answer closely resembling the real answer, it generates the output sequence in a loop, feeding the output from the previous time step to the decoder at each subsequent time step. The generator employs a complete transformer model comprising N identical layers and heads. Initially, the generator module undergoes independent training using Maximum Likelihood Estimation to enhance convergence

probability. Subsequently, the model undergoes fine-tuning via an adversarial network to learn the distribution of answers. The linear transformation and Gumbel SoftMax function are employed to convert the decoder's output into predicted next-token probabilities. The Gumbel- SoftMax distribution, known for approximating samples from a categorical distribution, facilitates the generation of an index vector with length L. This index vector serves as input to the discriminator. In the adversarial phase, the generator is updated upon interacting with the discriminator model. In a GAN model, this update is achieved by computing the gradient likelihood ratios of the objective function. LG represents the generator's loss function, m is the number of generated sequences, and Qi denotes the question regarded as conditioning data. As discussed earlier, the model employs the WGAN approach to address challenges associated with GANs and enhance training stability. Consequently, the WGAN objective function L_{wG} can be computed as described below.

$$L_{wG} = \nabla_{\theta_G} \left[\frac{1}{m} \sum_{i=1}^m f(G(Q_i)) \right]$$

Here, "f" must be a 1-Lipschitz function.

5.2 Discriminator Modules

The discriminator comprises the encoder portion of the transformer model, followed by a classifier. Its primary function is to assign probabilities to answers being real or fake at time step t , over the course of k epochs. In the GAN model, the discriminator is updated by computing the gradient likelihood ratios of the objective function. Here, LD represents the discriminator's loss function, A represents the real answer, and Q_i corresponds to the question used as conditioning data. In the WGAN model, the discriminator serves as a critic model and the loss function LC can be calculated. During each iteration step, the discriminator is trained twice: once with fake data pairs and once with real data pairs. Real question and answer pairs are tokenized and indexed using a pretrained BERT model. Due to the computational complexity of graph data, BERT is not used for embedding. Instead, the index matrix of the fake answer is concatenated with the index matrix of the question to form fake pairs. Subsequently, a linear model is applied to reduce the input matrix and sentence embedding's dimensionality. The output of the linear model is then combined with a positional encoding matrix.

6 Method

In the absence of recurrent neural networks (RNN) in the transformer architecture, which would inherently preserve sequential information and relative word positions, positional encoding becomes imperative to capture such relationships. This encoding step occurs subsequent to the conversion of each token into its respective embedding vector. For instance, if I have a sequence of tokens such as 23, 38, 1, 0, 0, the embedding layer will transform this tokenized sentence into a 5×100 -dimensional matrix, where 100 represents the length of the embedding vectors. To incorporate positional information, sine and cosine functions are utilized. The final positional encoding results in a 5×100 matrix. In the context of this research paper, I adopt the positional encoding method described herein.

6.1 Pad Mask

In the context of input sequences, a pad mask is employed to identify masked tokens. For instance, given the sequence 23, 38, 1, 0, 0, the pad mask would appear as 0, 0, 0, 1, 1, where '1' indicates masked tokens. During the computation of self-attention, tokens marked as '1' are excluded from the calculation, as they are considered padding tokens.

6.2 Lookahead Mask

The lookahead mask specifically pertains to the decoder section of the transformer architecture. Since words are generated sequentially, i.e., one word at a time, the lookahead mask ensures that self-attention is not computed for words that have not yet been generated. For instance, when predicting the first word, self-attention is not computed for the second word since it has not been generated. This masking continues as subsequent words are generated.

6.3 Self-Attention Calculation

Central to the transformer's distinctive capabilities is the concept of self-attention, represented as a matrix transformation. Given an input sequence, such as 23, 38, 1, 0, 0, the self-attention matrix takes the form of a 5x5 matrix, where each word's attention to other words in the same sentence is computed (hence, the term 'self-attention'). Importantly, self-attention is masked for padded tokens. The softmax operation, illustrated above, ensures that highly negative values corresponding to padded tokens render them effectively ignored in the calculation. The 'Q,' 'K,' and 'V' denote the encoded values of the input, projected into a lower dimension (referred to as 'depth'). The input, e.g., a 10x100 matrix, is passed through three separate dense layers, one for 'Q,' one for 'K,' and one for 'V.'

6.4 Multi-Head Attention Layer

The multi-head attention layer facilitates the calculation of self-attention multiple times, yielding various types of attention for each word in a sentence. In this paper, I adopt

the practice of calculating self-attention eight times. The code implementation of the multi-head attention layer may appear complex at first glance, but it is conceptually straightforward. It is treated as one of the transformer's layers, inheriting attributes from `tf.keras.layers`. This layer involves the creation of multiple weight matrices for attention calculation during the forward pass. Custom layer attributes include three dense layers, each dedicated to encoding 'Q,' 'K,' and 'V,' as well as one final dense layer for aligning the output shape with the input shape, `num heads` indicating the number of attention heads, and `d model` representing the model's dimensionality. It is essential to ensure that `d model` is divisible by `num heads` to achieve multi-head attention using this code implementation. During the forward pass (through the `call()` function), `'v,' 'k,' 'q,'` and `'mask'` are presented as parameters. These parameters correspond to input data. For instance, if the dimensions of the input embedded batch are 128x10x512 (BATCH, input seq length, embedding dimension), each of these parameters takes on this dimensionality. After processing these parameters through their respective dense layers, the `split heads()` function reshapes the data from 128x10x512 to 128x8x10x64.

The resulting shape (128x8x10x64) can be understood as follows: '128' represents the batch size, '8' signifies the number of attention heads, '10' indicates the length of the input sentence, and '64' represents the encoding of each word's embedding vector into a lower dimension (depth). This encoding corresponds to 'Q,' 'K,' and 'V' encodings in lower dimensions. Consequently, I obtain 'Q,' 'K,' and 'V' with shapes of 128x8x10x64 each. These values are utilized in the self-attention calculation, following the same transformation equations as before. The result is self-attention weights of dimensions 128x8x10x10 and an attention matrix measuring 128x8x10x64. To align this output shape with the input matrix, I concatenate the '8' attention heads along the last dimension (64). After concatenation, the shape becomes 128x10x512, and the data is subsequently passed through a dense layer comprising 512 units. The final output matrix assumes dimensions of 128x10x512, while the attention weight matrix measures 128x8x10x10. This detailed explanation elucidates the inner workings of the multi-head self-attention calculation, revealing that the transformer's power lies in its elegant simplicity.

6.5 Encoder

One of the intriguing aspects of the transformer architecture is the encoder, which essentially comprises a series of encoder layers. The number of times this encoder layer is repeated is, in fact, a hyperparameter that is subject to the complexity of the specific problem being tackled. In this particular implementation, I've opted for 2 repetitions. Within the encoder, I initiate the process by generating embeddings and subsequently incorporating positional encodings. After this preparatory step, I diligently invoke the encoder layer. A noteworthy aspect is the looping mechanism, where I set the `self.enc` layers attribute accordingly. This architecture, like others in the transformer, adheres to the convention of inheriting from the `tf.keras.layer` class. It's crucial to recognize that the output from the encoder becomes the input for the decoder, thus constituting a pivotal component of the overall transformer model.

6.6 Decoder Layer

The decoder layer exhibits a slight divergence from the encoder layer, primarily in the way attention is calculated. The fundamental 'self-attention' mechanism remains unchanged. Self-attention computations always pertain to words within the same sentence, whereas standard attention calculations, akin to a vanilla encoder-decoder with an attention mechanism, involve words from two distinct sentences. Consequently, the decoder layer incorporates two multi-head attention mechanisms: 'self' and 'standard'. In essence, it resembles the encoder layer with the inclusion of `mha1` and `mha2`. Notably, `mha1` takes 'answers' as input along with a lookahead mask to disregard self-attention calculation for future words. Following this, I observe the customary residual addition and layer normalization. On the other hand, `mha2` utilizes the encoder's output for its 'V' and 'K' dense layers and the output from `mha1` as its 'Q' dense layer. Therefore, `mha2` abstains from performing 'self-attention'. In the equation $(Q \cdot \text{transpose}(K)) \cdot V$, 'Q' originates from the decoder, while 'K' and 'V' emanate from the encoder's output. To illustrate, if the encoder input has a length of 7 and the decoder input spans 15 tokens, the resultant attention weight matrix will assume dimensions of 15x7, whereas the self-attention weight will manifest as 15x15. The

divergence between the encoder and decoder layer essentially boils down to 'mha2' and the application of the lookahead mask.

6.7 Decoder

In a manner akin to the encoder, the decoder calls the `DecoderLayer` multiple times. It entails the creation of 'answers' embeddings followed by the addition of positional encodings. The decoder layer is invoked seamlessly, adhering to the standard practice of inheriting from the `tf.keras.Layer` class. A crucial point to note is that with each repetition of the decoder layer, the encoder's output is provided (as input to `mha2`), and the self-attention and standard attention weights produced by each decoder layer (from `mha1` and `mha2`) are retained in an attention weights dictionary. This data will be instrumental for plotting attentions. Finally, the output matrix is returned for word prediction.

6.8 Transformer

The overall architecture of the transformer can be succinctly described as follows: Encoder call leads to decoder call, which in turn leads to a dense layer, ultimately resulting in word tokens. The final dense layer encompasses a vocabulary size matching that of 'answers,' producing a tensor of dimensions $128 \times 10 \times 512$. The tensor emerging from the decoder, after passing through this final dense layer, assumes dimensions of $128 \times 10 \times 27360$ (tokenizer `a.vocab` size + 2). To provide structure to the model, I've derived the `Transformer` class from `tf.keras.Model`. These 11 steps culminate in the realization of a powerful framework that holds the potential to execute most natural language processing tasks with a degree of human-like understanding. The elegance of this approach lies in its simplicity and versatility. For additional insights into custom optimizers, loss functions, and detailed training and inference procedures, I invite you to explore the accompanying notebook, where you'll find comprehensive explanations and code implementations.

7 Results

For Model-1 which is the initial model, designated as Model-1, boasted 12 million trainable parameters and was trained over 25 epochs. However, this model displayed limited learning capabilities, which was unsurprising given that transformers typically thrive on extensive trainable parameters to achieve optimal predictions. Regrettably, the predictions generated by Model-1 were entirely nonsensical and not fit for sharing.

For Model-2, a significantly more sophisticated architecture, featured a formidable 24 million trainable parameters and underwent a rigorous training regimen of 350 epochs. It exhibited slow yet continued learning even beyond the 350-epoch mark, owing to the relatively small number of parameters. The results obtained with Model-2, although not initially impressive in terms of loss and accuracy up to epoch 350, were remarkably satisfying. The replies generated by this model outshone its predecessors, leaving the previous two models in the dust. This achievement was quite astonishing, considering that my model appeared minuscule for such a complex task. This section outlines the results of the generative Chatbot model's performance, offering insights into the dataset used and a comparative analysis with other methodologies. It commences with an explanation of the implementation specifics, followed by a brief introduction to the two datasets employed for evaluation. Subsequently, the evaluation metrics utilized in the assessment process are delineated. Finally, the paper presents a comparative analysis of the proposed architecture against state-of-the-art models.

7.1 Evaluation Metrics

The proposed model's performance is evaluated using a range of metrics, including BiLingual Evaluation Understudy (BLEU), Recall-Oriented Understudy for Gisting Evaluation (ROUGE-L), F-measure, and Metric for Evaluation of Translation with Explicit ORdering (Meteor). Each metric offers unique insights into the model's performance, evaluating aspects like precision, recall, compromise between recall and precision, and similarity to human judgments.

7.2 Experimental Results

The effectiveness of the proposed architecture is demonstrated on both the Cornell Movie Dialogs Corpus and Chit-Chat datasets, utilizing BLEU4, ROUGE-L, F-measure, and Meteor metrics for evaluation. Notably, the proposed model exhibits superior performance on the Chit-Chat dataset, a reflection of the dataset’s conversational nature compared to the movie dialogue-based Cornell dataset.

8 Discussion

The primary challenge faced by most contemporary Chatbot approaches lies in their sequential nature, which often results in less accurate outcomes and suboptimal performance. The proposed model addresses this challenge by employing a hybrid of cWGAN (conditional Wasserstein Generative Adversarial Network) and a transformer model. The experimental results have shown that the model excels in generating highly accurate and semantically relevant responses for Chatbot dialogues. However, the model’s ability to diversify its generated responses is somewhat constrained due to the inherent characteristics of the training dataset. Both datasets utilized for training lack a multitude of responses for each question. Here, I delve into the proposed model’s analysis from two distinct perspectives:

8.1 Analysis of Generator Pretraining

Before initiating adversarial training in the proposed model, the generator model undergoes a separate training phase powered by a transformer model. This pretraining phase significantly impacts the overall performance of the model. In the best efforts, I execute 200 cycles to train the transformer as a pretrained model for the generator. Subsequently, the proposed model undergoes adversarial training for 400 cycles. As illustrated in Table 7, the model trained solely with the transformer, without incorporating adversarial learning, exhibits the lowest performance. I leverage this model as a pretrained foundation for initiating the adversarial training phase. The results clearly demonstrate that this approach enhances the Chatbot’s performance across all metrics on the Cornell dataset.

8.2 Analysis of Loss Function

The choice of loss functions plays a pivotal role in the performance of deep models. In this context, I delve into the loss functions employed for both the generator and discriminator during the adversarial training phase. Instead of a conventional GAN model, the adversarial training phase utilizes a Wasserstein GAN (WGAN), which leverages a new cost function based on the Wasserstein distance. This distance metric offers a smoother gradient and trains independently of the generator's implementation. Calculating the Wasserstein distance necessitates finding a Lipschitz function, which can be learned using a deep network. This network closely resembles the discriminator but lacks a sigmoid function and yields a scalar score rather than a probability. This score essentially quantifies the authenticity of the input data and serves as the "critic."

The generator's loss function initially remains low due to the presence of a pretrained generator model. After a few epochs, the loss function experiences an increment, followed by a gradual reduction below its initial value. This phenomenon signifies the model's ability to grasp the data distribution during adversarial learning. In parallel, the discriminator begins with a low loss function. However, as the generator learns the data distribution and generates more authentic fake data, the discriminator's loss function steadily increases. Ultimately, both the generator and discriminator loss functions converge. In conclusion, the proposed model represents a significant advancement in the field of Chatbot development. It overcomes the limitations of sequential approaches, offering highly accurate responses. Furthermore, the discussions encompass the critical aspects of generator pretraining and the role of loss functions, shedding light on the model's remarkable performance improvements. The utilization of a Wasserstein GAN and the transformation of data distribution through adversarial learning have proven instrumental in enhancing the Chatbot's capabilities.

9 Conclusion and Future Direction

In this study, I introduced an innovative model that combines the strengths of the cWGAN (conditional Wasserstein Generative Adversarial Network) and the transformer model. The proposed model comprises two integral components: the Generator and the Discriminator. The Generator leverages a complete Transformer model, while the Discriminator incorporates only the encoder segment of a transformer model, followed by a classifier. I conducted a comprehensive evaluation of the proposed model across two diverse datasets, employing a range of evaluation metrics. The outcomes unequivocally establish the superiority of the proposed model when compared to state-of-the-art approaches, as demonstrated through the BLEU4, ROUGE-L, F-measure, and Meteor metrics. By harnessing the capabilities of cWGAN in tandem with the transformer model, the proposed approach excels in generating responses that are not only accurate but also possess semantic relevance and exhibit human-like qualities. Looking ahead, future endeavors may explore the integration of reinforcement learning techniques to further enhance the semantic associations between questions and answers across various domains. This avenue holds promise for advancing the field of natural language processing and facilitating even more refined and context-aware conversational agents.

10 Code

10.1 Data Preparation

```
tokenizer a = tfds.deprecated.text.SubwordTextEncoder.build f
    rom corpus(train[ 'answer' ], target vocab size=2**15)

tokenizer q = tfds.deprecated.text.SubwordTextEncoder.build f
    rom corpus(train[ 'question' ], target vocab size
        =2**15)

def encode(ques, ans):
    ques = [tokenizer q.vocab size] + tokenizer q.encode(ques.numpy()) +
    [tokenizer q.vo

    ans = [tokenizer a.vocab size] + tokenizer a.encode(ans.numpy()) + [t
    okenizer a.voca

    return ques, ans

def tf_encode(ques, ans):
    result ques, result ans = tf.py function(encode, [ques, ans], [tf.int
    64, tf.int64])result ques.set shape([None])
    result ans.set shape([None])
    return result ques, result ans

train dataset = tf.data.Dataset.from tensor slices(dict(train))
train dataset = train dataset.map(lambda x:tf_encode(x[ 'question' ]
, x[ 'answer' ]))train dataset = train dataset.cache()

train dataset = train dataset.shuffle(20000).padded batch(64, padded
shapes=([None],[Non

train dataset = train dataset.prefetch(tf.data.experimental.AUTOTUNE)

val dataset = tf.data.Dataset.from tensor slices(dict(validation))
val dataset = val dataset.map(lambda x:tf_encode(x[ 'question' ]
, x[ 'answer' ]))val dataset = val dataset.padded batch(64,
```

```
padded_shapes=([None],[None]))
```

10.1 Positional Encoding

```
def get_angles(pos, i, d_model):
```

```
    angle_rates = 1 / np.power(10000, (2 * (i//2)) / np.float32(d_model))
```

```
    return pos * angle_rates
```

```
def positional_encoding(position, d_model):
```

```
    angle_rads = get_angles(np.arange(position)[:, np.newaxis], - np.arange (
```

```
    d_model)[np.newaxis angle_rads[:, 0::2] = np.sin(angle_rads[:, 0::2])
```

```
    angle_rads[:, 1::2] = np.cos(-angle_rads[:,
```

```
    1::2])pos_encoding = angle_rads[np.newaxis
```

```
    ,:]
```

```
    return tf.cast(pos_encoding, dtype=tf.float32)
```

10.2 Pad Mask

```
def create_padding_mask(seq):
```

```
    seq = tf.cast(tf.math.equal(seq, 0), tf.float32)
```

```
    return seq[:, tf.newaxis, tf.newaxis, :]
```

10.3 Lookahead Mask

```
def create_look_ahead_mask(size):
```

```
    mask = 1 - tf.linalg.band_part(tf.ones((size, size)), -1, 0)
```

```
    return mask # (seq_len, seq_len)
```

10.4 Self-Attention Calculation

```
def scaled_dot_product_attention(q, k, v,
                                mask=None):
    matmul_qk = tf.matmul(q, k,
                           transpose_b=True)
    dk = tf.cast(tf.shape(k)[-1], tf.float32)
    scaled_attention_logits = matmul_qk / tf.math.sqrt(dk)
    if mask is not None:
        scaled_attention_logits += (mask * -1e9) # -
    attention_weights = tf.nn.softmax(scaled_attention_logits,
                                     axis=-1)
    output = tf.matmul(attention_weights, v)

    #

    return output, attention_weights
```

10.5 Encoder

```
class Encoder(tf.keras.layers.Layer):
    def __init__(self, num_layers, d_model, num_heads, dff, input_vocab_size,
               maximum_position_encoding):
        super(Encoder, self).__init__()
        self.d_model = d_model
        self.num_layers = num_layers
        self.embedding = tf.keras.layers.Embedding(input_vocab_size, d_model)
        self.pos_encoding = positional_encoding(maximum_position_encoding,
                                                self.d_model)

        self.enc_layers = [EncoderLayer(d_model, num_heads, dff, rate) for
                           in range(num_layers)]

        self.dropout = tf.keras.layers.Dropout(rate)

    def call(self, x, training,
            mask):
        seq_len = tf.shape(x)[0]
```

```

(x)[1]
x = self.embedding(x) # (batch_size, input_seq_len, d_model)
x *= tf.math.sqrt(tf.cast(self.d_model, tf.
float32)) x += self.pos_encoding[:, :seq le
n, :]
x = self.dropout(x, training=training)
for i in range(self.num_layers):    x = self.enc_layers[i](x, tra
ining, mask)

return x # (batch_size, input_seq_len, d_model)

```

References

- [1] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, Jeffrey Dean *Distributed Representations of Words and Phrases and their Compositionality* .
- [2] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, Sameer Singh (2015) *Beyond Accuracy: Behavioral Testing of NLP Models with CheckList*
- [3] Quoc Le, Tomas Mikolov (2014) *Distributed Representations of Sentences and Documents*
- [4] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever (2018) *Language Models are Unsupervised Multitask Learners*
- [5] Tyler A. Chang, Zhuowen Tu, Benjamin K. Bergen (2022) *The Geometry of Multilingual Language Model Representations* , arXiv
- [6] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, Laurent Sifre (2022) *Training Compute-Optimal Large Language Models* , arXiv

